香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

# String Algorithms

- Naive Matching
- Rabin-Karp
- Boyer-Moore
- KMP

Yanlin Zhang | DSAA 2043 Spring 2026

# Agenda



- Goal: solve exact string matching efficiently.
- Key question: after a mismatch, what information can still survive?
  - naive matching
  - Rabin-Karp
  - Boyer-Moore
  - KMP

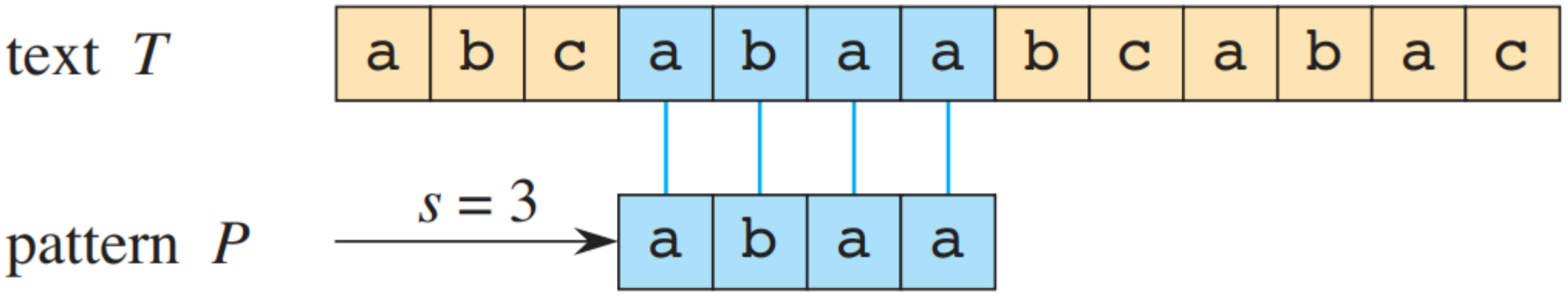
# Applications of String Matching



- Editors and IDEs need responsive search while a document is still changing.
- Bioinformatics searches short patterns inside extremely long DNA strings.
- Search and log analysis may reuse one pattern on many different texts.

# Exact String Matching: Problem Statement

- Input: a text T[1..n] and a pattern P[1..m].
- Output: all shifts s such that T[s + 1 .. s + m] = P[1 .. m].



# Prefix and Suffix



- A prefix is a beginning part of a string;


- a suffix is an ending part.

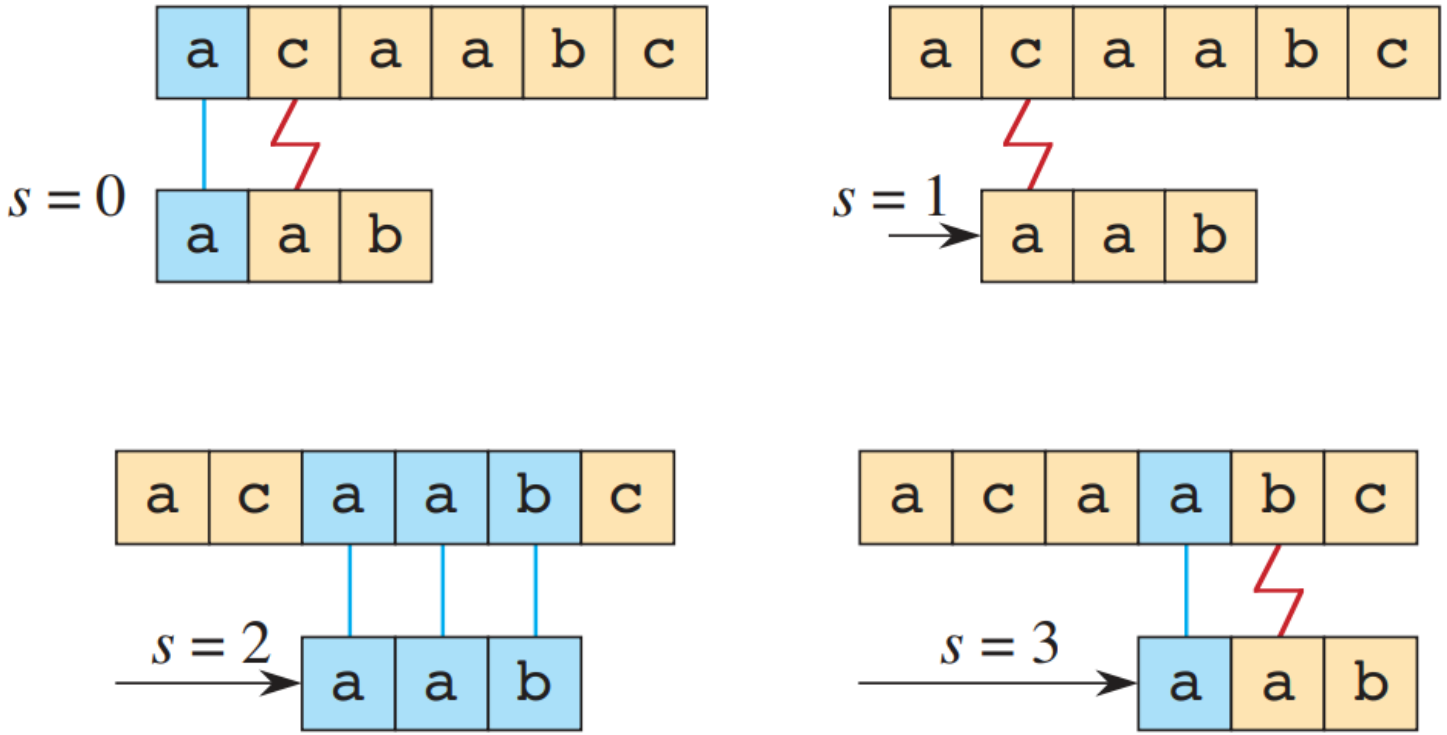
# Naive Matcher: Check Every Shift

- Slide the pattern over the text one shift at a time.
- At shift s, compare P with T[s + 1 .. s + m] from left to right.
- After a mismatch, the next shift starts again from P[1].

```
NAIVE-STRING-MATCHER(T, P)
1   n = T.length
2   m = P.length
3   for s = 0 to n − m
4       if P[1..m] == T[s + 1..s + m]
5           print "Pattern occurs with shift" s
```

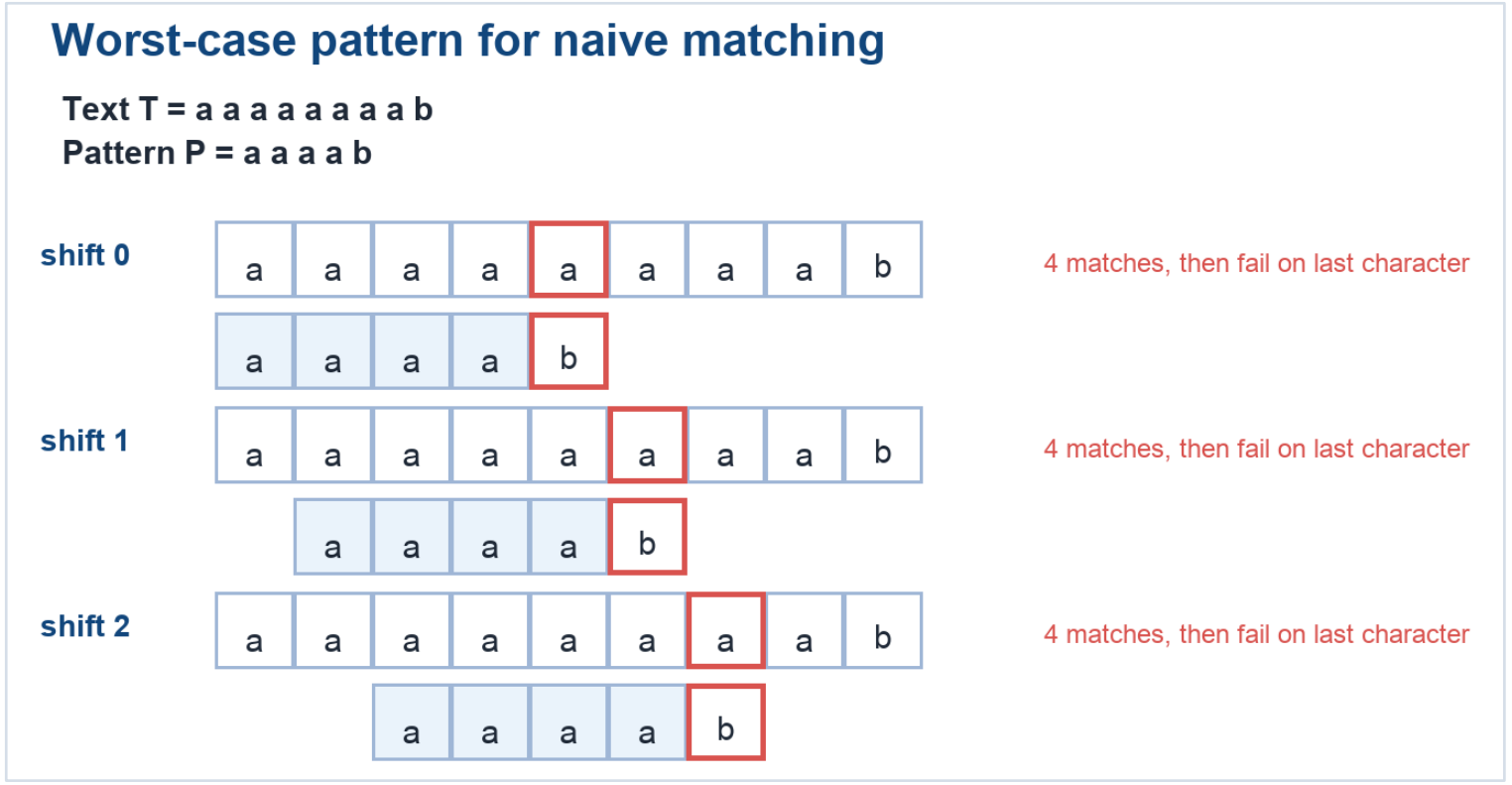
# Naive Matching Example

- After each failed shift, the pattern simply moves one position to the right.
- Nothing from the previous partial match is remembered.



# Naive Matching: Running Time

- There are n - m + 1 candidate shifts.
- One shift may compare as many as m characters before failing.
- So the worst case running time is O((n - m + 1)m).
- How about the average case?



# Rabin-Karp: Fingerprint Idea

- Interpret each length-m string as a number.
- Let p be the fingerprint of P and let t(s) be the fingerprint of window T[s + 1 .. s + m].
- If p ≠ t(s), reject shift s immediately; if p = t(s), verify characters.
    - Rabin-Karp uses fingerprints as a filter

**At one shift s**

pattern P → fingerprint p

text window T[s+1..s+m] → fingerprint t_s

p ≠ t_s → impossible shift

p = t_s → candidate shift; verify characters

# Fingerprint: Covert a string to a base-d digit

香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

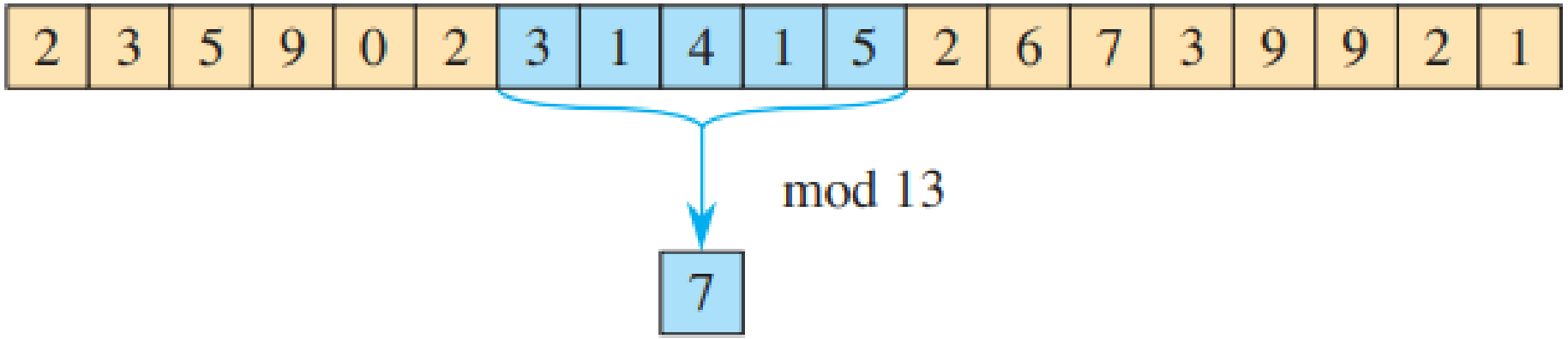
- Given a string S, we can convert it to a number in base-d

$$H(S) = (s_0 \cdot d^{m-1} + s_1 \cdot d^{m-2} + \cdots + s_{m-1} \cdot d^0) \bmod q$$

| Step | Component | Example: String "abc" |
|---|---|---|
| **Characters** | $s_0, s_1, s_2$ | `'a'`, `'b'`, `'c'` |
| **Integer Values** | $v_i$ (ASCII/Map) | $1, 2, 3$ |
| **Base Power** | $d^{m-1-i}$ (Weight) | $10^2, 10^1, 10^0$ |
| **Positional Value** | $v_i \cdot d^p$ | $100, 20, 3$ |
| **Final Sum** | $\sum$ | **123** |
| Last step | mod q | 123%q |

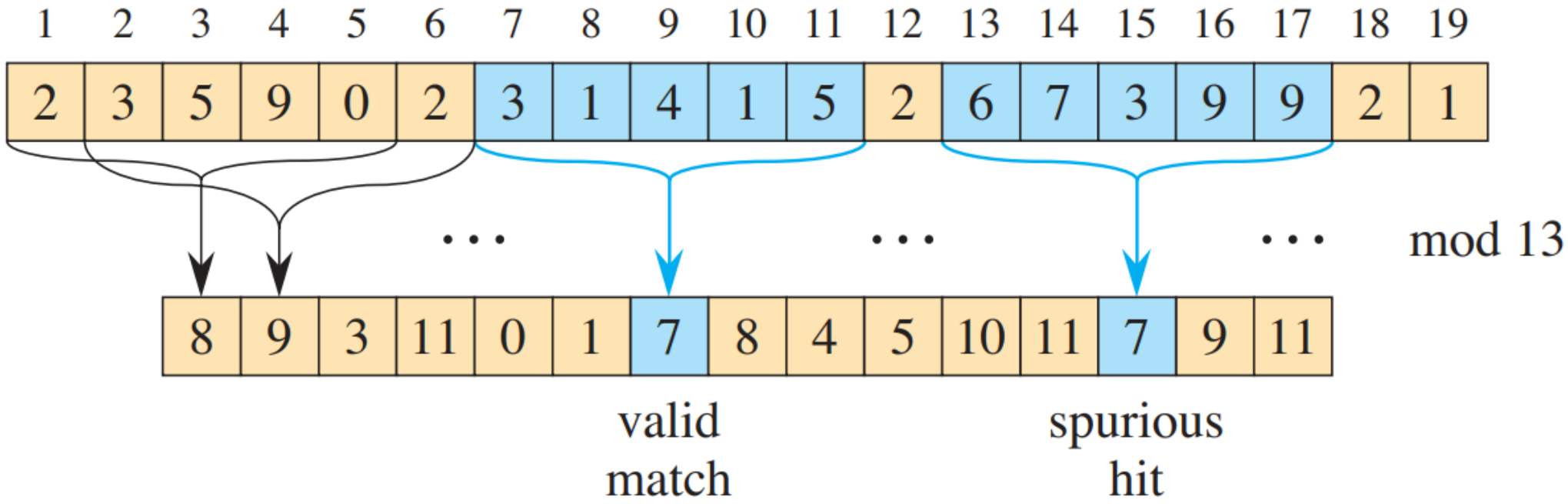
# Example: Fingerprint of a Window

- Preprocessing computes the pattern fingerprint p and the per window fingerprint $t_i$.
- Matching starts from these two numbers rather than from m direct character comparisons.



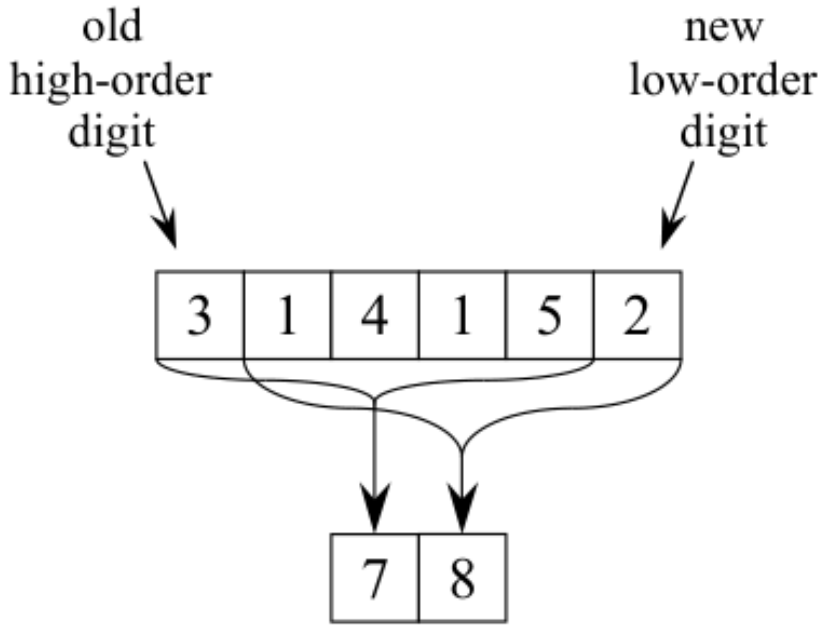
# Example: Real Hit and Spurious Hit

- Fingerprint equality only says that a shift is a candidate.
- A candidate may be real or spurious, so Rabin-Karp must still verify characters.



# O(1) Rolling Update

- The old window fingerprint is reused directly.
- One digit leaves, the remaining digits shift left, and one new digit enters.




$$
\begin{aligned}
14152 &\equiv (31415 - 3\cdot 10000)\cdot 10 + 2 \pmod{13} \\
&\equiv (7 - 3\cdot 3)\cdot 10 + 2 \pmod{13} \\
&\equiv 8 \pmod{13}
\end{aligned}
$$

(old high-order digit; shift; new low-order digit)

# Rabin-Karp



```
RABIN-KARP-MATCHER(T, P, n, m, d, q)
 1  h = d^(m−1) mod q
 2  p = 0
 3  t_0 = 0
 4  for i = 1 to m                          // preprocessing
 5      p = (dp + P[i]) mod q
 6      t_0 = (dt_0 + T[i]) mod q
 7  for s = 0 to n − m                      // matching—try all possible shifts
 8      if p == t_s                         // a hit?
 9          if P[1 : m] == T[s + 1 : s + m] // valid shift?
10              print "Pattern occurs with shift" s
11      if s < n − m
12          t_{s+1} = (d(t_s − T[s + 1]h) + T[s + m + 1]) mod q
```

# Why Verification Is Necessary in Rabin-Karp

香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

- If two strings are equal, their fingerprints are equal, so a real match is never filtered out.
- But equal fingerprints do not guarantee equal strings, because spurious hits can occur. → How?

- That is why Rabin-Karp verifies characters before reporting a shift.
- Correctness still comes from direct comparison; fingerprints only speed up filtering.

# Rabin-Karp: Running Time

- Rolling the fingerprint from t_s to t_{s+1} costs O(1).
- Expected speed comes from good filtering: most shifts satisfy p ≠ t_s and are rejected immediately.
- Worst case time appears when many equal fingerprints still force full verification.

| Case | What happens | Time |
|---|---|---|
| Typical | With a good modulus, spurious hits stay rare and most shifts satisfy p ≠ t_s. | Expected O(n + m) |
| Worst case | Many shifts satisfy p = t_s and trigger full character verification. | O((n - m + 1)m) |

# The Boyer-Moore (BM) Algorithm

香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

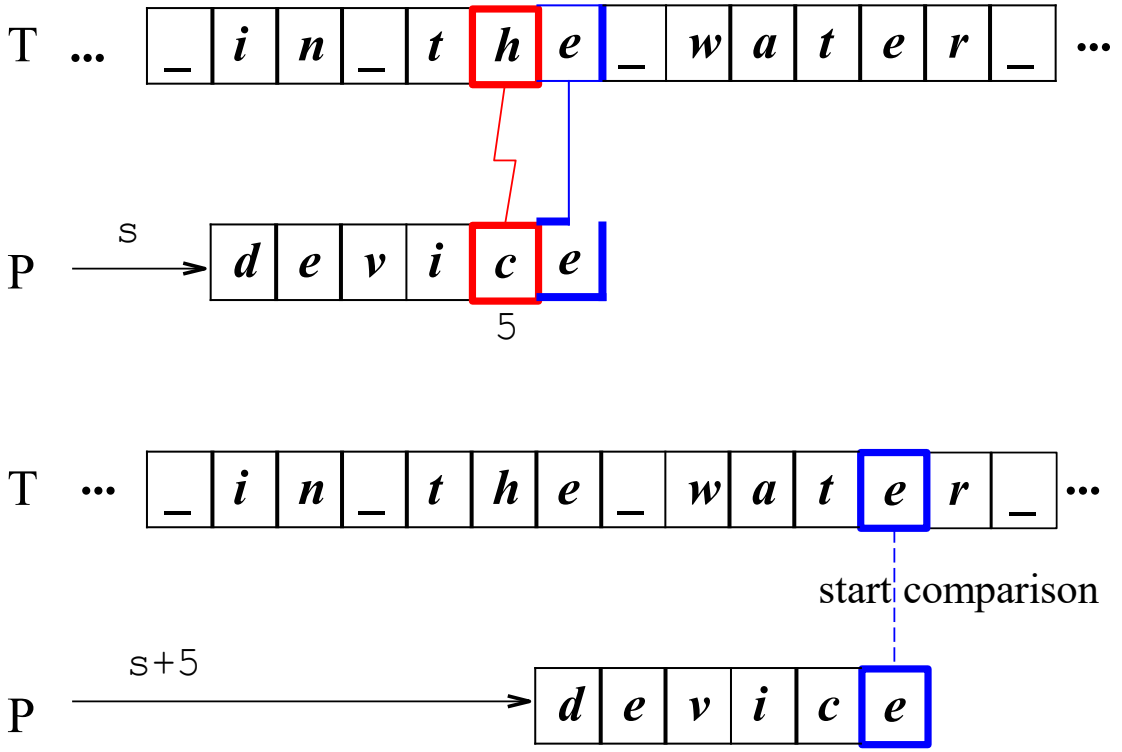
- The Boyer-Moore (BM) algorithm slides $P$ from left to right; however it compares $P$ and $T$ **from right to left**, i.e., $P[m]$ will first compare with $T[i]$. If they match, it then compares $P[m-1]$ with $T[i-1]$, etc. Else, it slides $P$ to right, and compare $P[m]$ with $T$ again.

# The Boyer-Moore (BM) Algorithm

香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

## The BM Algorithm : the bad-character heuristic

One insight of BM algorithm is that, if there is a mismatch between $P[j]$ $and$ $T[i]$, and $T[i]$ does not appear in $P$. $P$ should be advanced by $j$.

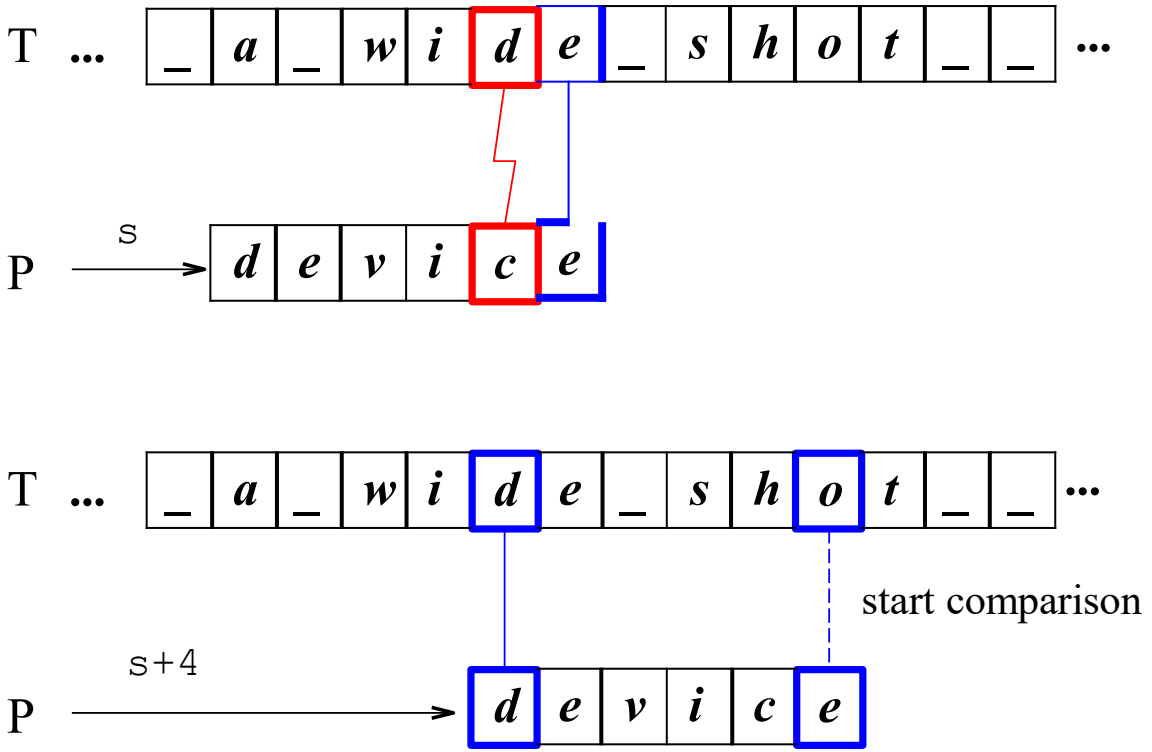


# The Boyer-Moore (BM) Algorithm

香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

**The BM Algorithm : the bad-character heuristic**

If $T[i]$ appears in $P$, shift $P$ such that $T[i]$ is aligned with the *rightmost* occurrence of $T[i]$ in $P$.

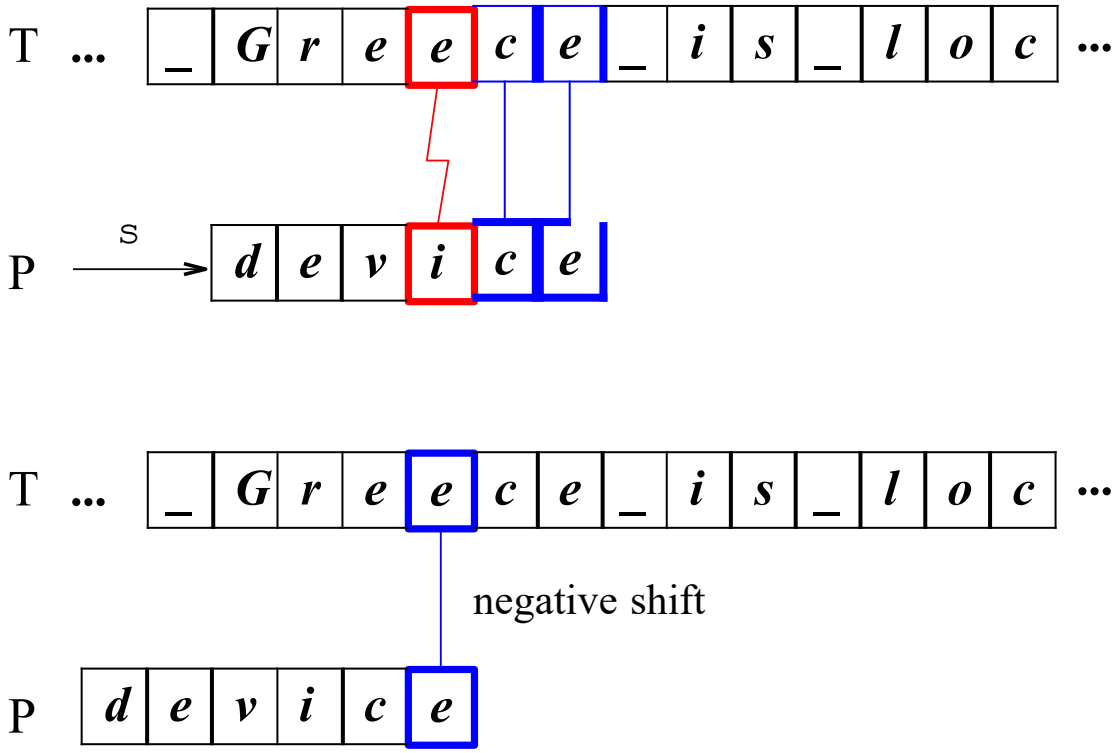


# The Boyer-Moore (BM) Algorithm

香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

## The BM Algorithm : the bad-character heuristic

If it happens the alignment of $T$ and $P$ gives a *negative* shift value, then just ignore it.

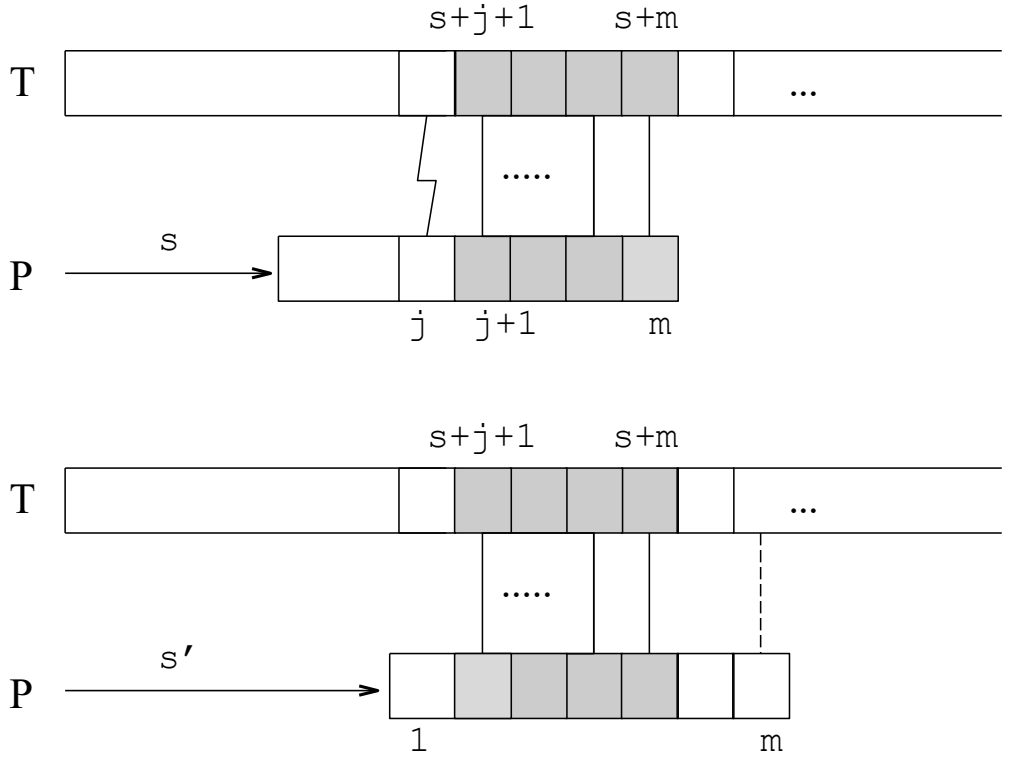


# The Boyer-Moore (BM) Algorithm

香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

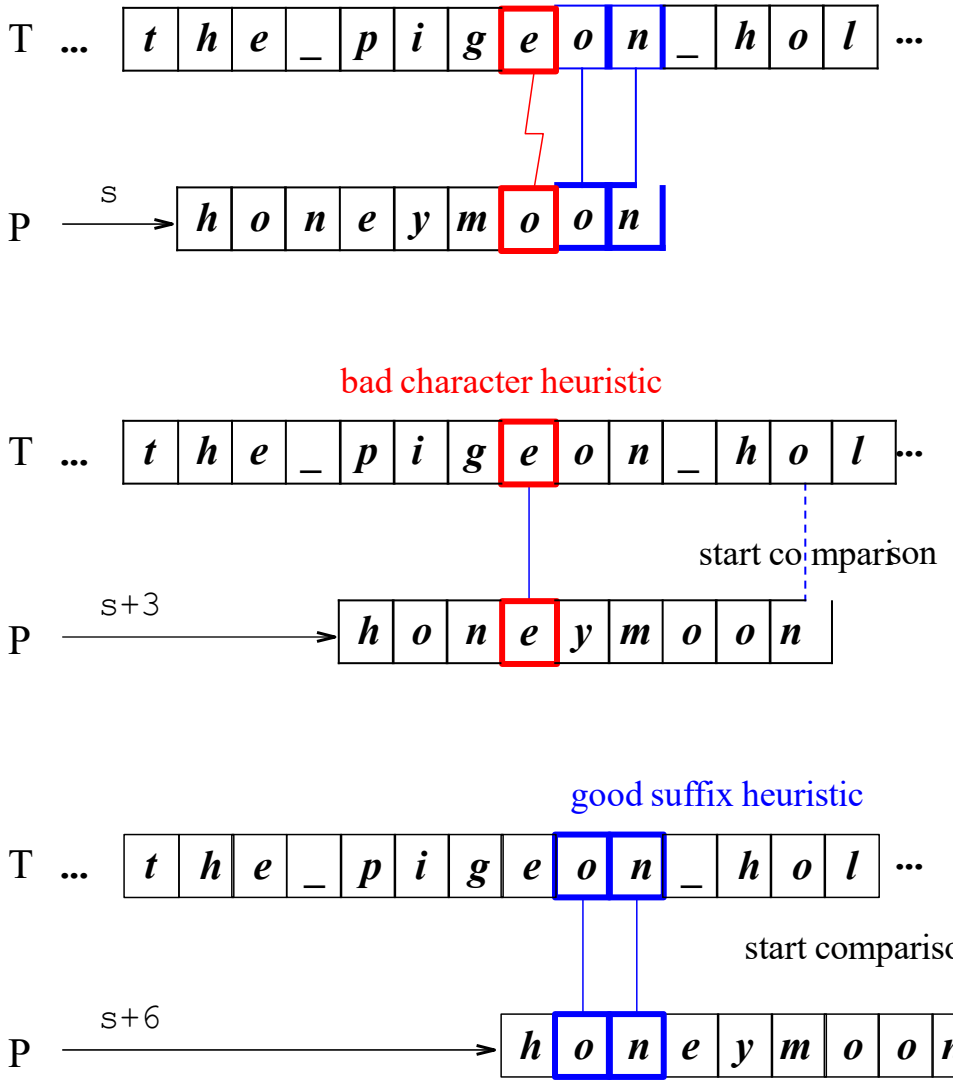
## The BM Algorithm : the good suffix heuristic

If the current shift is $s$, and it is a mismatch at $P[j]$ , then we know $t = P[j+1..m] = T[s+j+1..s+m]$. Then we can shift $P$ by $s$' such that $T$ is aligned with the rightmost occurrence (other than ) of $t$ itself.



# The Boyer-Moore (BM) Algorithm

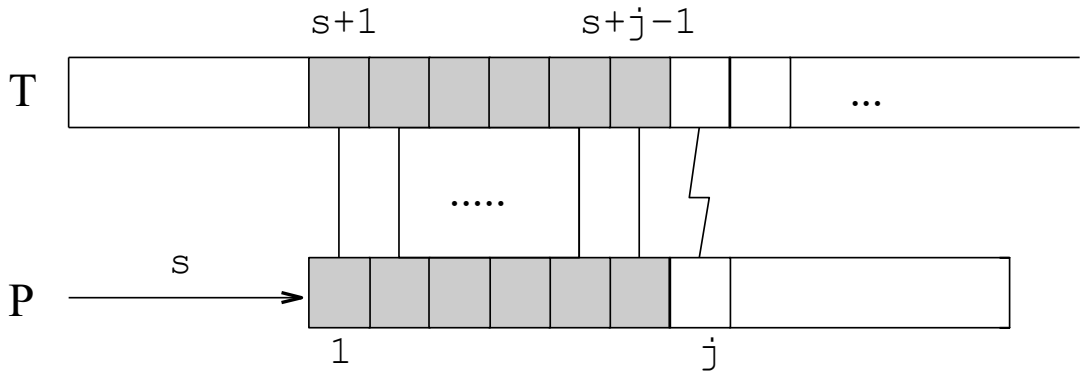
The BM Algorithm takes the larger shift amount computed by bad-character heuristic and good-suffix heuristic.



# The Knuth-Morris-Pratt (KMP) Algorithm

In the Brute-Force algorithm, if a mismatch occurs at $P[j]$ $(j > 1)$, it only slides $P$ to right by 1 step. It throws away one piece of information that we've already known. What is that piece of information?

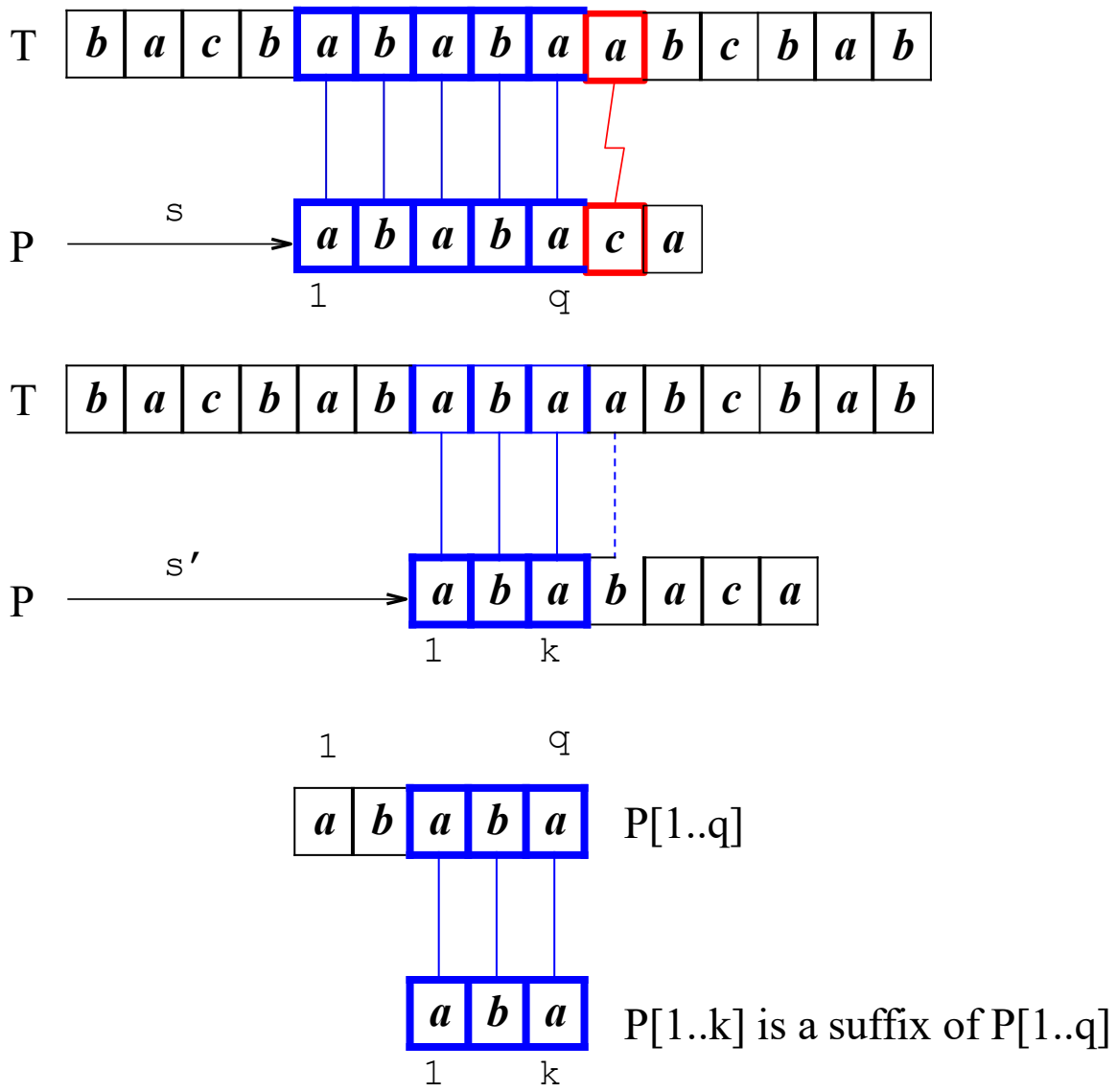
Let $s$ be the current shift value. Since it is a mismatch at $P[j]$ (recall we compare from left to right), we know $T[s + 1..s + j - 1] = P[1..j - 1]$.




How can we make use of this information to make the next shift? In general, $P$ should slide by $s' > s$ such that $P[1..k] = T[s' + 1..s' + k]$. We then compare $P[k + 1]$ with $T[s' + k + 1]$.

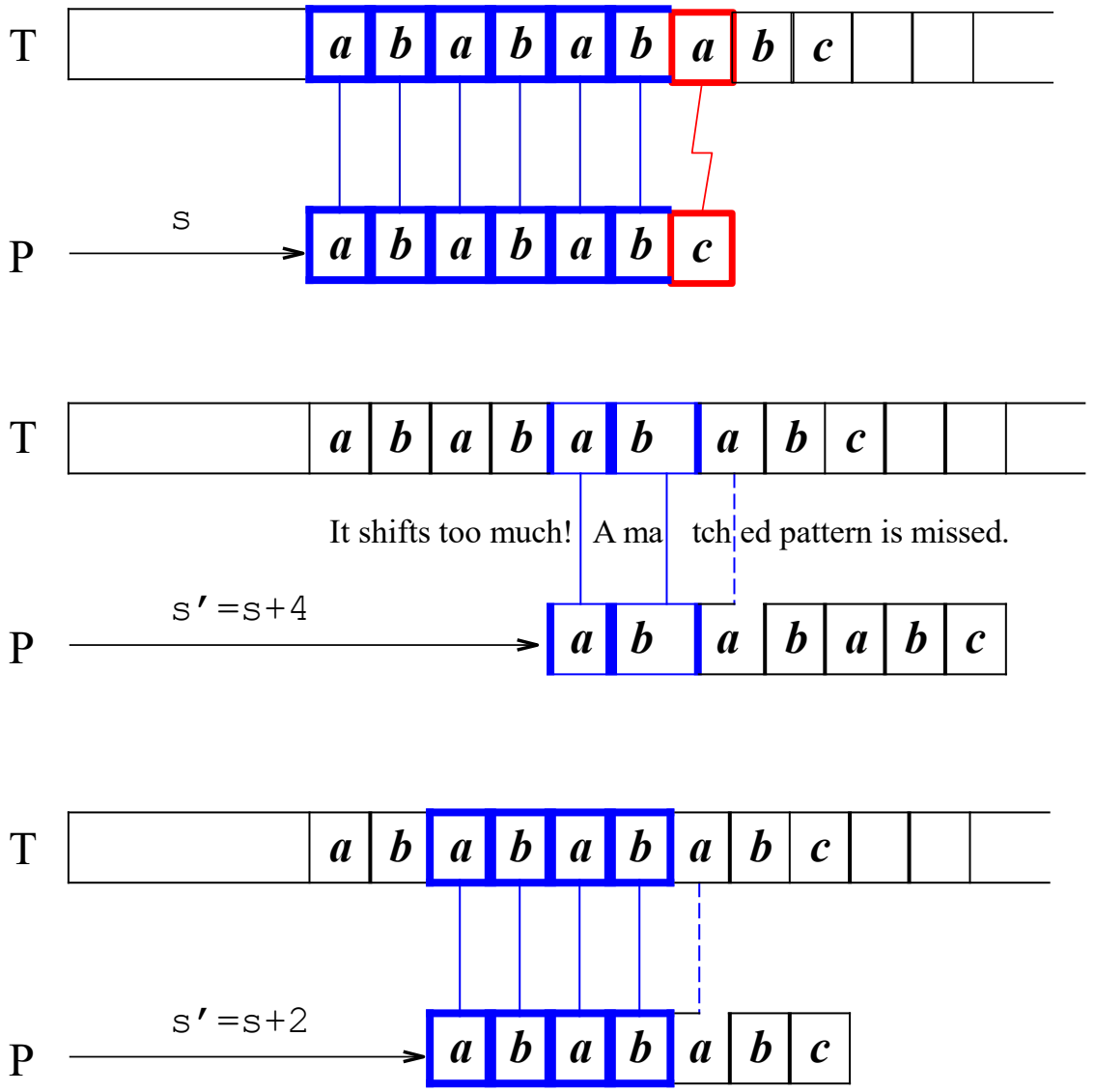
# The Knuth-Morris-Pratt (KMP) Algorithm

When we slide $P$ to right, it should be a place where $P$ could possibly occur in $T$.



# The Knuth-Morris-Pratt (KMP) Algorithm

**Do not shift too much:** Do not shift too much, as it may miss some matched patterns!



# The Knuth-Morris-Pratt (KMP) Algorithm

香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

**The *next* function :**

We need to answer the following question: Given $P[1 \ldots q]$ match text characters $T[s+1 \ldots s+q]$, what is the least shift $s' > s$ such that

$$P[1 \ldots k] = T[s'+1 \ldots s'+k]$$

where $s' + k = s + q$?

In practice, the shift $s'$ can be precomputed by comparing $P$ against itself. Observe that $T[s'+1 \ldots s'+k]$ is a known text, and it is a **suffix** of $P[1 \ldots q]$. To find the least shift $s' > s$, it is the same as finding the largest $k < q$, s.t.,

$P[1 \ldots k]$ is a suffix of $P[1 \ldots q]$.

# The Knuth-Morris-Pratt (KMP) Algorithm

香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

**The *next* function :**

Given $P[1 \ldots m]$, let $next$ be a function $\{1,2, \ldots, m\} \to \{0,1, \ldots, m-1\}$ such that $next(q) = \max\{k : k < q \text{ and } P[1 \ldots k] \text{ is a suffix of } P[1 \ldots q]\}$.

| q | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| P[q] | ***a*** | ***b*** | ***a*** | ***b*** | ***a*** | ***b*** | ***a*** | ***b*** | ***c*** | ***a*** |
| next(q) | 0 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 0 | 1 |

Given $next(q)$ for all $1 \leq q \leq$ m, we can use the KMP algorithm.

# The Knuth-Morris-Pratt (KMP) Algorithm

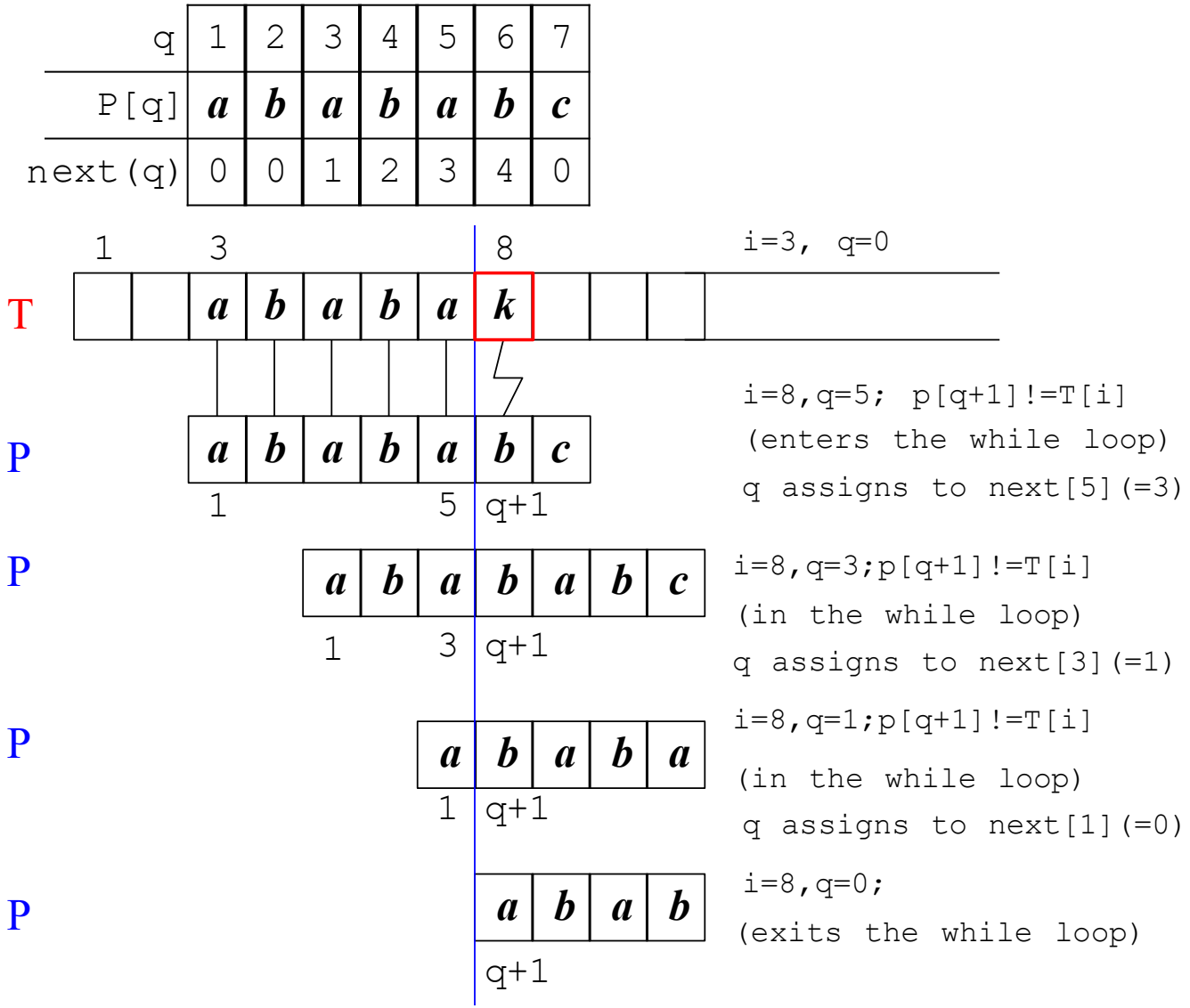


```
KMP_StringMatcher(T, P) {
 n = length(T); m = length(P); compute_Next(P);
 q = 0; // number of characters matched so far
 i=1;
 while (i<=n) {
  // loop until a match is found, or
  // number of characters matched so far
  // is 0; note 'i' is unchanged.
  while (q > 0 and P[q+1] != T[i]) { q=next[q];
  }
  // matched character increased by 1
  if (P[q+1]==T[i]) q=q+1;
  if (q==m) {
   print "Pattern occurs with shift=", i-m
   q=next[q];
  }
  i++;
 }
}
```

Running time is O(n):

- If mismatch happens (P[k+1] != P[q]), pattern P slides to the right by at least one;
- Otherwise, i increases by one.

# The Knuth-Morris-Pratt (KMP) Algorithm



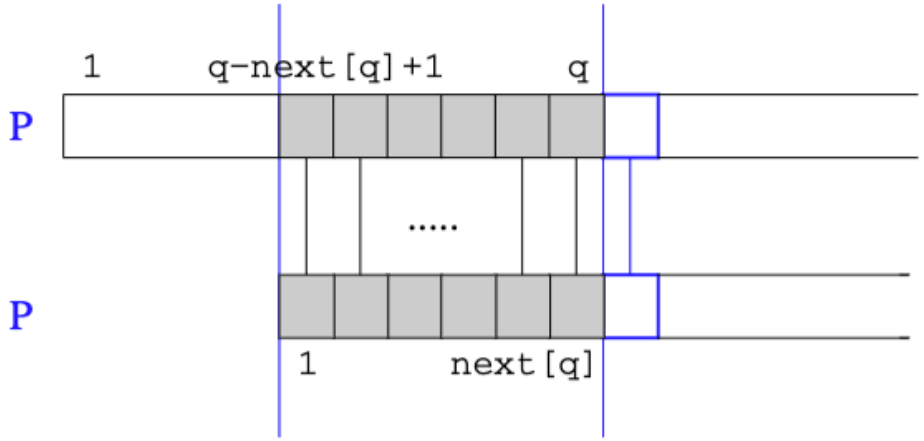
# The Knuth-Morris-Pratt (KMP) Algorithm

香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

**How to compute the *next* function :**

Given $next[1],\ next[2], \ldots,\ next[q]$, how to compute $next[q+1]$?
Assume that we have a pattern P and a text P.

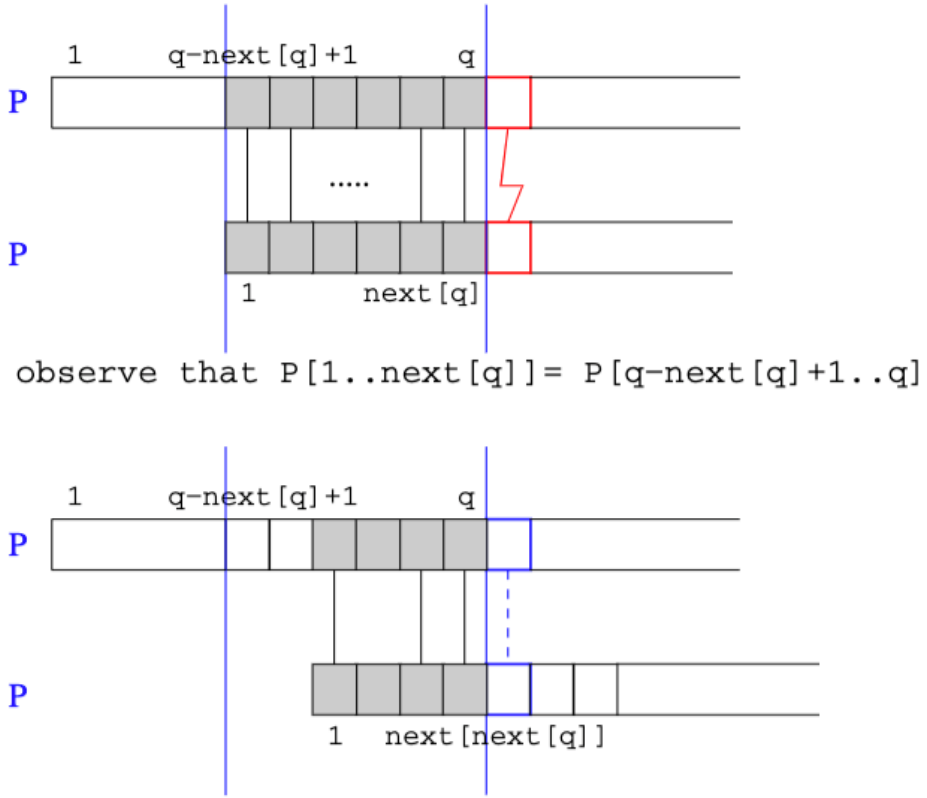
1. If $p[q+1] == p[next[q]+1]$,
   then $next[q+1] = next[q] + 1$.



# The Knuth-Morris-Pratt (KMP) Algorithm

2. **If** $P[q+1] != P[next[q]+1]$, **then do what?**

$P$ should slide to a place such that the prefix of $P$ $[1..next[q]]$ occurs as a suffix of $P$ $[q-next[q]+1..q]$; this information is stored in $next[next[q]]$ !



# The Knuth-Morris-Pratt (KMP) Algorithm



We first set $next[1] = 0$, then compute $next[q]$ with $q = 2, 3, \ldots m$, one by one in $m - 1$ iterations.

```
compute_Next(P) {
 m = length(P);
 next[1]=0; // initialization
 k = 0; // number of characters matched so far
 q=2;
 while (q<=m) {
  while (k > 0 and P[k+1] != P[q]) { k = next[k];
  }
  if (P[k+1]==P[q]) k=k+1;
  next[q]=k;

  q++;
 }
}
```

Running time is O(m):

- If mismatch happens (P[k+1] != P[q]), pattern P slides to the right by at least one;
- Otherwise, q increases by one.

# The Knuth-Morris-Pratt (KMP) Algorithm

香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)

- **Running Time of the KMP Algorithm:**
  - Computing next function: O(m);
  - Compute all the occurrences: O(n);
  - The total running time is therefore O(n + m).